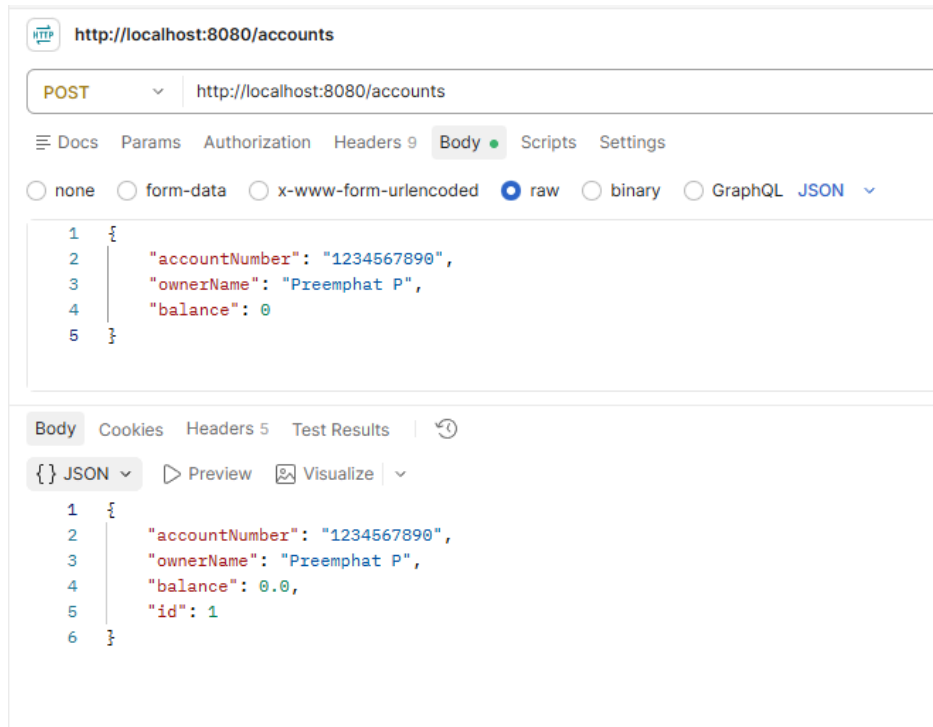


Lab 9

1 สร้าง Account

คำอธิบาย : เป็นการสร้างบัญชีใหม่ พอเราส่งเลขบัญชีกับชื่อไป Spring Boot ก็จะเอาข้อมูลนี้ไปสร้างลงตาราง account ใน Database แล้วส่งข้อมูลบัญชีที่สร้างเสร็จกลับมาให้เราดูว่าได้ id : 1



2.ฝากเงิน

อธิบาย: เป็นการสั่งฝากเงิน 1,000 บาท เข้าบัญชี id 1 พอเรายิง Request ไป ตัว DepositService จะทำงาน 2 อย่างพร้อมกัน คือ บวกเงินเพิ่มในบัญชี และบันทึกประวัติการฝากเงิน พอทำเสร็จทั้งคู่ก็จะตอบกลับมาว่า "Deposit successful" แปลว่าฝากเงินเรียบร้อยแล้ว

HTTP **http://localhost:8080/accounts/1/deposit**

POST **http://localhost:8080/accounts/1/deposit**

Docs Params Authorization Headers 9 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   |   "amount": 1000
3 }
```

Body Cookies Headers 5 Test Results ↺

{ } **JSON** ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   |   "message": "Deposit successful"
3 }
```

3.ตรวจสอบ Account

อธิบาย : เป็นการดึงข้อมูลบัญชี id 1 อีกรอบ จะเห็นว่ายอดเงิน (balance) เปลี่ยนจาก 0 เป็น 1000 บาท แล้ว แสดงว่าโค้ดฝากเงินที่เราทำไปมันอัปเดตยอดเงินได้จริง

HTTP **http://localhost:8080/accounts/1**

GET **http://localhost:8080/accounts/1**

Docs Params Authorization Headers 7 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary

1

Body Cookies Headers 5 Test Results ↺

{ } **JSON** ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   |   "accountNumber": "1234567890",
3   |   "ownerName": "Preemphat P",
4   |   "balance": 1000.0,
5   |   "id": 1
6 }
```

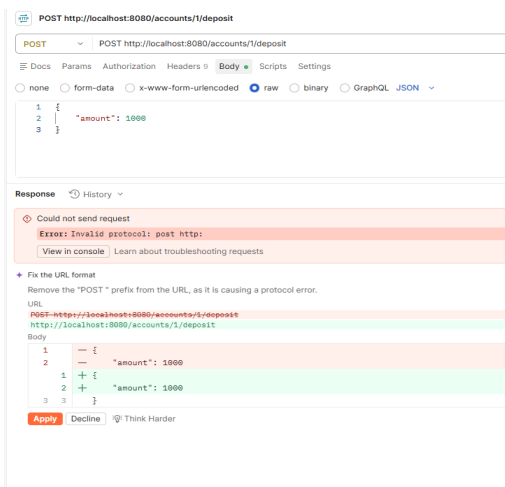
4.ตรวจสอบ DeposiTransaction

Data Output			
Messages			
Notifications			
	id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1

อธิบาย : พอเปิดดูตาราง deposit_transaction ใน pgAdmin จะเห็นสลิป/ประวัติการฝากเงินเพิ่มขึ้น มา 1 แถว ยอดเงิน 1,000 บาท ผูกกับ account_id = 1 เป็นการยืนยันว่าข้อมูลถูกเซฟลง Database PostgreSQL จริงๆ ไม่ได้หายไปไหน

5.ทดลอง Rollback

อธิบาย เมื่อเกิด Exception กลางคัน Annotation @Transactional จะสั่งให้ระบบ(Rollback)ข้อมูล ทั้งหมดที่ทำไปก่อนหน้าทันที ทำให้ยอดเงินในบัญชีไม่ถูกบวกเพิ่ม และไม่มีประวัติการฝากเงินถูกบันทึก ค้างไว้ ทำให้ข้อมูลในระบบถูกต้องและไม่พัง



public.deposit_transaction/lab9/postgres@PostgreSQL 18

Query

Query History

1

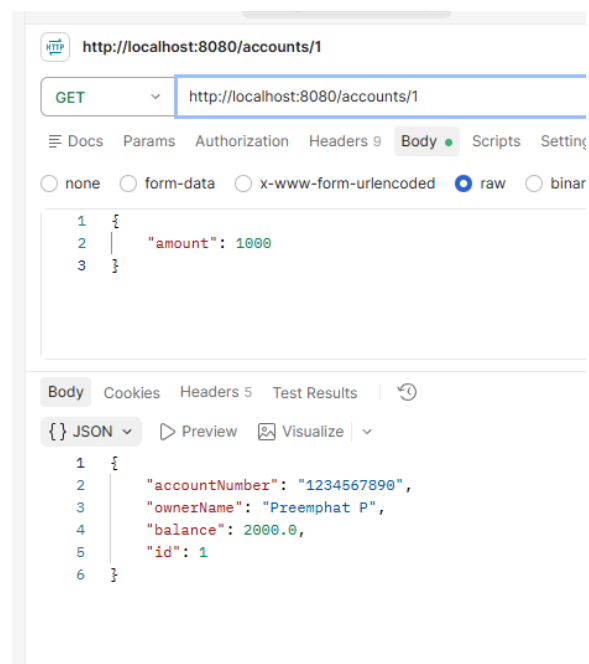
SELECT * FROM public.deposit_transaction

2

ORDER BY id ASC

public.account/lab9/postgres@PostgreSQL 18

อธิบาย เมื่อเราเอา @Transactional ออก พอโปรแกรมทำงานถึงคำสั่งอัปเดตยอดเงิน คำสั่งนั้นจะถูกบันทึกลงฐานข้อมูลทันที ทำให้ยอดเงินเปลี่ยนเป็น 2,000 บาท แต่เมื่อโค้ดบรรทัดถัดมาเกิด Error พังกลางคัน ระบบจะ ไม่มีการ Rollback ยอดเงินกลับ ทำให้ประวัติการฝากเงินในตาราง deposit_transaction ไม่ถูกสร้าง ส่งผลให้เกิดปัญหา ข้อมูลพังและไม่สอดคล้องกัน คือเงินถูกเพิ่มไปแล้วแต่ไม่มีหลักฐานประวัติการฝากเงิน



1. @Transactional มีหน้าที่อะไร?

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

ตอบ เพราะประกอบด้วยการทำงานมากกว่า 1 ขั้นตอน (เพิ่มเงิน + บันทึกสลิป) ต้องสำเร็จพร้อมกันทั้งหมด เพื่อป้องกันข้อมูลไม่สอดคล้องกัน

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

ตอบ COMMIT: ยืนยันการบันทึกข้อมูลลงฐานข้อมูลอย่างสมบูรณ์

ROLLBACK: ยกเลิกการเปลี่ยนแปลงข้อมูลทั้งหมด และย้อนกลับไปจุดก่อนเริ่มทำงาน

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

ตอบ One-to-Many (1 บัญชี มีได้หลายประวัติการฝากเงิน) หรือมองจากฝั่ง DepositTransaction จะเป็น Many-to-One

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ตอบ ไม่เปลี่ยนแปลง ข้อมูลจะถูก Rollback กลับไปอยู่ในสถานะเดิมก่อนเริ่มทำธุรกรรม

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

ตอบ เกิด Data Inconsistency ยอดเงินในบัญชีถูกเพิ่มไปแล้ว แต่ประวัติการฝากเงินไม่ถูกบันทึกเนื่องจากโปรแกรมพังกลางคันโดยไม่มีการ Rollback